🏛 GLA UNIVERSITY, MATHURA — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

HPC Masterclass | Day 3 In-Class Checkpoint Assessment

Course: B.Tech CSE 7th Sem — High Performance Computing Masterclass
Module: Day 3 — Hardware-Aware Deep Learning, Intel oneDNN, Slurm Workload Manager & Horovod
Duration: 30 Minutes | **Total Questions**: 25 MCQs | **Maximum Marks**: 25

Student Name: _____ **Roll Number**: _____ **Section**: _____

📝 Instructions for Candidates

1. All questions are compulsory. Each question carries exactly 1 mark.
2. Select the single best answer for each question.
3. No negative marking.

Q01. In enterprise HPC clusters, what is the primary fundamental difference between running machine learning workflows on a dedicated cluster versus a standard single workstation?

- ▢ **(a)** HPC clusters only run interpreted Python without native C++ compilation.
- ▢ **(b)** HPC clusters decouple compute and storage across distributed nodes connected via ultra-low-latency fabrics (e.g., InfiniBand/RoCE) and leverage parallel file systems (e.g., Lustre/GPFS).
- ▢ **(c)** HPC clusters do not support GPU acceleration and rely exclusively on single-threaded CPUs.
- ▢ **(d)** HPC workstations automatically eliminate the need for batch queuing systems like Slurm.

Q02. In hardware-aware deep learning optimization, how are standard neural network layers fundamentally categorized regarding hardware execution constraints?

- ▢ **(a)** Static vs. Dynamic layers.
- ▢ **(b)** Compute-bound layers (e.g., GEMM, Conv2D) and Memory-Bandwidth-bound layers (e.g., LayerNorm, Softmax, ReLU).
- ▢ **(c)** Single-precision layers vs. Double-precision layers only.
- ▢ **(d)** Scalar layers vs. Microcontroller layers.

Q03. What is Intel oneDNN (formerly MKL-DNN), and what role does it serve in deep learning software stacks?

- ▢ **(a)** A high-level GUI for annotating image datasets.
- ▢ **(b)** An open-source, cross-platform performance library of basic deep learning primitives optimized for Intel architecture CPUs and GPUs.
- ▢ **(c)** A proprietary compiler replacing GCC and Clang for operating system kernels.
- ▢ **(d)** A specialized database engine for storing neural network weights.

Q04. How does oneDNN operator fusion (e.g., fusing Conv2D + BiasAdd + ReLU) achieve substantial performance speedups during neural network execution?

- ▢ **(a)** By increasing the total number of floating-point operations executed.
- ▢ **(b)** By retaining intermediate activation tensors inside fast L1/L2 CPU caches or registers, eliminating redundant write-backs and reads from high-latency main memory (DRAM).
- ▢ **(c)** By compressing neural network weights to 1-bit representations automatically.
- ▢ **(d)** By converting all multi-threaded operations into single-threaded serial loops.

Q05. Which environment variable explicitly forces modern TensorFlow builds to enable oneDNN graph optimizations and custom execution kernels on Intel CPUs?

- ▢ **(a)** OMP_NUM_THREADS=1
- ▢ **(b)** TF_ENABLE_ONEDNN_OPTS=1
- ▢ **(c)** INTEL_MKL_DISABLE=1
- ▢ **(d)** SLURM_NNODES=4

Q06. Why do Intel oneDNN primitives convert standard tensor layouts (like NHWC or NCHW) into blocked formats such as 'nChw16c' or 'nChw8c' internally on x86_64 processors?

- ▢ **(a)** To ensure compatibility with 32-bit legacy x86 machines.
- ▢ **(b)** To align the channel dimension with hardware SIMD register widths (e.g., 512-bit registers holding 16 single-precision floats) for contiguous vector load/store instructions.
- ▢ **(c)** To encrypt the tensor data in system RAM.
- ▢ **(d)** To force all matrix computations to run on the GPU instead of the CPU.

Q07. What is Intel AMX (Advanced Matrix Extensions) introduced in 4th Gen Intel Xeon Scalable processors (Sapphire Rapids), and how does it accelerate deep learning?

- ▢ **(a)** A software-only Python module for matrix transposition.
- ▢ **(b)** A hardware silicon architecture featuring 2D register files (Tiles) and a Tile Matrix Multiply (TMUL) accelerator designed for high-throughput BF16 and INT8 matrix operations.
- ▢ **(c)** An optical network interconnect card operating at 800 Gbps.
- ▢ **(d)** A PCIe expansion bridge for legacy VGA displays.

Q08. What architectural capability characterizes the Intel Data Center GPU Max Series (formerly code-named Ponte Vecchio) for HPC and Deep Learning?

- ▢ **(a)** It utilizes ARM architecture with integrated SATA controllers.
- ▢ **(b)** It features Xe-HPC architecture with Xe Core vector and matrix (XMX) engines, massive HBM2e memory bandwidth (up to 3.28 TB/s), and Xe Link inter-GPU fabrics.
- ▢ **(c)** It is an FPGA designed exclusively for software network emulation.
- ▢ **(d)** It requires software emulation to process standard FP32 floating point data.

Q09. Which software package enables standard TensorFlow applications to transparently target Intel Data Center GPUs (XPUs) via oneAPI SYCL runtimes?

- ▢ **(a)** Intel Extension for TensorFlow (ITEX)

- ▢ **(b)** Intel MPI Benchmarks (IMB)
- ▢ **(c)** Gnuplot 5.4
- ▢ **(d)** Slurm Job Scheduler

Q10. What is the primary compute architecture of Habana Gaudi deep learning processors developed by Intel?

- ▢ **(a)** A standard x86 CPU with hyperthreading.
- ▢ **(b)** A heterogeneous architecture combining VLIW SIMD Tensor Processor Cores (TPCs) with dedicated GEMM Matrix Multiplication Engines (MMEs).
- ▢ **(c)** A RISC-V microcontroller cluster running at 50 MHz.
- ▢ **(d)** A fixed-function ASIC strictly limited to linear regression.

Q11. What unique networking integration distinguishes Habana Gaudi processors (e.g., Gaudi 2 / Gaudi 3) from conventional discrete GPUs?

- ▢ **(a)** They rely solely on external USB 3.0 dongles for inter-node communication.
- ▢ **(b)** They integrate 24 standard 100GbE RoCE (RDMA over Converged Ethernet) ports directly onto each OAM processor silicon substrate, enabling all-to-all scaling without proprietary NICs.
- ▢ **(c)** They transmit training data over analog audio cables.
- ▢ **(d)** They require dedicated Ethernet switches inside every individual PCIe slot.

Q12. What is the role of Intel Habana SynapseAI software suite in distributed deep learning?

- ▢ **(a)** It is a desktop video editing utility.
- ▢ **(b)** It provides the graph compiler, runtime engine, communication libraries (HCL), and framework plugins for PyTorch and TensorFlow targeting Gaudi processors.
- ▢ **(c)** It is an unmanaged hypervisor for KVM virtual machines.
- ▢ **(d)** It replaces standard Linux kernel system calls with DOS commands.

Q13. In a Slurm-managed HPC cluster, what is the role of the 'slurmctld' daemon?

- ▢ **(a)** It executes tasks directly on worker compute node CPU cores.
- ▢ **(b)** It is the central management daemon that monitors cluster resources, manages job queues, and allocates nodes to pending jobs.
- ▢ **(c)** It compiles C and Fortran code into machine binaries.
- ▢ **(d)** It formats hard drives when a job completes.

Q14. Which Slurm batch script directive specifies that a distributed deep learning job requires exactly 4 physical nodes?

- ▢ **(a)** #SBATCH --time=04:00:00
- ▢ **(b)** #SBATCH --nodes=4
- ▢ **(c)** #SBATCH --partition=debug
- ▢ **(d)** #SBATCH --mem=4G

Q15. What does the Slurm command 'squeue' perform when executed by an HPC engineer?

- ▢ **(a)** It deletes all user files in the /tmp partition.

- ▢ **(b)** It queries and displays the status of jobs currently in the scheduling queue (running, pending, or completing).
- ▢ **(c)** It reboots all compute nodes in the cluster immediately.
- ▢ **(d)** It prints the temperature of the server room air conditioning.

Q16. Which environment variable is automatically injected by Slurm into a running job script to indicate the total number of allocated compute nodes?

- ▢ **(a)** SLURM_NNODES
- ▢ **(b)** OMP_NUM_THREADS
- ▢ **(c)** CUDA_VISIBLE_DEVICES
- ▢ **(d)** PYTHONPATH

Q17. Why is the '#SBATCH --exclusive' flag frequently used when submitting multi-node deep learning benchmark jobs to Slurm?

- ▢ **(a)** To prevent anyone from reading the final research paper.
- ▢ **(b)** To ensure allocated nodes are dedicated 100% to the job, preventing CPU, RAM, and PCIe bandwidth contention from other tenants' noisy-neighbor jobs.
- ▢ **(c)** To run the job exclusively on the master head node.
- ▢ **(d)** To permanently reserve the nodes even after the job terminates.

Q18. How does Horovod differ fundamentally from traditional Parameter Server (PS) architectures in distributed deep learning?

- ▢ **(a)** Horovod requires a dedicated central master server that stores all parameters and collects all gradients.
- ▢ **(b)** Horovod utilizes a decentralized Ring-AllReduce algorithm over MPI/NCCL, where all workers communicate peer-to-peer and compute aggregate gradients collaboratively.
- ▢ **(c)** Horovod only runs on single-core mobile processors.
- ▢ **(d)** Horovod discards gradients instead of averaging them across workers.

Q19. What is the primary network scalability bottleneck of the Parameter Server (PS) architecture when scaling deep learning training to 64+ nodes?

- ▢ **(a)** The PS server runs out of disk storage for log files.
- ▢ **(b)** The network bandwidth on the parameter server(s) scales linearly with the number of workers ($O(N)$), causing severe network saturation and idle worker stall.
- ▢ **(c)** Python garbage collection stops all network switches.
- ▢ **(d)** Ethernet packets cannot travel across more than 2 switches.

Q20. In the Ring-AllReduce algorithm utilized by Horovod, what is the total data volume transferred by each worker node to complete a full gradient synchronization of size M bytes across N workers?

- ▢ **(a)** $2 \times \frac{N-1}{N} \times M$
- ▢ **(b)** $N^2 \times M$
- ▢ **(c)** $\frac{M}{N^2}$
- ▢ **(d)** $M \times \log_2(N) \times N$

Q21. Which Horovod API function must be invoked at the very beginning of a distributed training script to initialize the communication runtime?

- ▢ **(a)** hvd.shutdown()
- ▢ **(b)** hvd.init()
- ▢ **(c)** hvd.broadcast_variables()
- ▢ **(d)** hvd.rank()

Q22. In a multi-GPU/XPU server running Horovod, why is it critical to bind each rank using 'hvd.local_rank()' rather than 'hvd.rank()'?

- ▢ **(a)** 'hvd.rank()' returns negative numbers on Linux.
- ▢ **(b)** 'hvd.local_rank()' provides the device index local to the physical node (e.g., 0 to 7), preventing multiple processes on the same host from contending for GPU 0.
- ▢ **(c)** 'hvd.local_rank()' disables GPU acceleration to save power.
- ▢ **(d)** 'hvd.local_rank()' causes all workers to run on CPU core 0.

Q23. In TensorFlow 2.x with Horovod, how is automatic gradient averaging across all distributed workers implemented?

- ▢ **(a)** By manually writing an MPI_Allreduce loop inside a C++ extension.
- ▢ **(b)** By wrapping the standard TensorFlow optimizer with 'hvd.DistributedOptimizer(optimizer)'.
- ▢ **(c)** By setting learning rate to zero.
- ▢ **(d)** By saving weights to disk after every batch and reloading them.

Q24. Why is 'hvd.broadcast_variables(model.variables, root_rank=0)' required before commencing the training loop in Horovod?

- ▢ **(a)** To print the neural network architecture to the console.
- ▢ **(b)** To ensure all worker ranks initialize with identical initial model weights and optimizer states from root rank 0, preventing divergent optimization paths.
- ▢ **(c)** To delete random weights from memory.
- ▢ **(d)** To upload model checkpoints to a public cloud bucket.

Q25. According to Goyal et al.'s Linear Scaling Rule in distributed deep learning, how should the base learning rate (LR_{base}) typically be adjusted when scaling batch size from a single worker to N workers?

- ▢ **(a)** $LR_{new} = LR_{base} / N$
- ▢ **(b)** $LR_{new} = LR_{base} \times N$
- ▢ **(c)** $LR_{new} = LR_{base} \times \sqrt{N}$
- ▢ **(d)** $LR_{new} = LR_{base} / \log(N)$